

# **MOSAIC 0.2 — Micro-skill Orchestration through Skill Activation and Instruction Composition: uma arquitetura mínima para compor micro skills sobre ferramentas gerais**

*MOSAIC 0.2 — Micro-skill Orchestration through Skill Activation and Instruction Composition: a minimal architecture for composing micro-skills over general-purpose tools*

**João Vitor Scheuermann**

MOSAIC 0.2 — 7 de agosto de 2026 / 7 August 2026

*Documento técnico de arquitetura. Esta versão especifica o núcleo conceitual e os contratos mínimos do MOSAIC; não apresenta implementação executável, benchmark ou resultados experimentais.*

## **RESUMO**

Este artigo especifica a MOSAIC 0.2 (Micro-skill Orchestration through Skill Activation and Instruction Composition), uma arquitetura mínima para agentes de linguagem que combinam micro skills, tools executáveis e a capacidade cognitiva do modelo. O planejamento é orientado a resultados; o roteamento usa o body integral de `SKILL.md`; o bundle ordenado contém zero, uma ou várias skills; e o menu combina tools base com aquelas declaradas pelo bundle. A revisão 0.2 separa explicitamente quatro objetos: a decisão semântica escrita pelo modelo, as `Observation` criadas pelo runtime, o `NodeOutcome` materializado pelo runtime ao anexar todas as observações ordenadas, e o pedido de revisão associado automaticamente ao conjunto completo de evidências do nó. Identificadores opacos do provedor servem apenas à correlação interna e nunca são reproduzidos pelo modelo nem enviados ao planner localizado. O perfil mantém catálogo canônico, feedback body-aware, execução orientada por estado, revisão localizada e composição final determinística, assumindo runtime confiável em modo YOLO. **Palavras-chave:** agentes de linguagem; micro skills; uso de ferramentas; planejamento; evidência de runtime; revisão localizada.

## **ABSTRACT**

This article specifies MOSAIC 0.2 (Micro-skill Orchestration through Skill Activation and Instruction Composition), a minimal architecture for language agents that combine micro-skills, executable tools, and the model’s cognitive capability. Planning is outcome-oriented; routing reads the full `SKILL.md` body; an ordered bundle contains zero, one, or several skills; and the visible menu combines base tools with tools declared by that bundle. Version 0.2 explicitly separates four objects: the model-authored semantic decision, runtime-created `Observation` records, the runtime-materialized `NodeOutcome` that attaches every ordered observation, and

a revision request automatically associated with the node's complete evidence set. Opaque provider identifiers are used only for internal correlation and are neither reproduced by the model nor exposed to the localized planner. The specification retains a canonical catalog, body-aware feedback, state-guided execution, localized revision, and deterministic final assembly under a trusted YOLO-mode runtime. **Keywords:** language agents; micro-skills; tool use; planning; runtime evidence; localized revision.

# 1 INTRODUÇÃO

## 1.1 Problema

Agentes baseados em modelos de linguagem costumam receber dois tipos de extensão. O primeiro é composto por instruções reutilizáveis que ensinam um modo de trabalhar. O segundo é composto por operações executáveis que permitem ler dados, calcular valores, criar artefatos ou produzir efeitos externos. Neste artigo, o primeiro tipo é chamado de *skill* e o segundo de *tool*. A distinção parece simples, mas frequentemente desaparece na arquitetura. Uma *skill* é tratada como se fosse uma API; uma *tool* é tratada como se contivesse o método completo para resolver uma tarefa; e o plano acaba sendo construído como uma sequência de chamadas. Esse desenho funciona para pedidos estreitos, como enviar uma mensagem pronta a um canal. Ele é menos adequado para tarefas abertas. Considere o pedido:

*Leia o último relatório financeiro, interprete os resultados e retorne a melhor oportunidade.*

O pedido contém pelo menos quatro tipos de trabalho: localizar o documento correto, disponibilizar seu conteúdo, interpretar evidências e comparar alternativas. Localização e leitura dependem de operações externas. Interpretação, síntese e decisão podem ser realizadas pelo modelo, desde que ele receba instruções adequadas. Criar *tools* artificiais como `interpret_financial_report` ou `choose_best_opportunity` apenas deslocaria para uma API o raciocínio que o modelo continuaria precisando executar. O problema tratado pelo MOSAIC é, portanto:

*Como planejar por resultados e compor várias instruções comportamentais sobre *tools* gerais ou específicas, sem exigir uma relação um-para-um entre objetivo, *skill* e *tool*?*

## 1.2 Princípio de redução

A arquitetura segue uma regra de design deliberada: um componente só pertence ao núcleo quando é necessário para expressar a hipótese central. É fácil acrescentar registros, políticas, resolvedores, verificadores e camadas de compatibilidade; o trabalho de projeto está em retirar aquilo que não altera o conceito. Esse critério produz duas consequências. Primeiro, o artigo especifica um núcleo pequeno, e não uma plataforma operacional completa. Segundo, elemen-

tos úteis em produção não são negados; apenas deixam de ser pré-requisitos conceituais. Uma implementação futura pode acrescentar segurança, isolamento, observabilidade, retries, provedores ou otimização, desde que haja uma razão concreta para isso.

### 1.3 Contribuições da especificação

A contribuição da MOSAIC não é um novo algoritmo de embeddings, um novo modelo de linguagem ou um novo protocolo de tool calling. A proposta é uma organização arquitetural com seis decisões centrais.

**Quadro 1 - Decisões centrais da MOSAIC 0.2**

| Decisão                                                                | Efeito arquitetural                                                                                                    |
|------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Planejar por objetivos orientados a resultados</b>                  | A granularidade do plano não copia tools nem arquivos de skills.                                                       |
| <b>Tratar skills como instruções e tools como operações</b>            | O modelo continua responsável por interpretação, transformação e decisão.                                              |
| <b>Selecionar bundles ordenados de zero a várias skills</b>            | Um objetivo combina comportamentos complementares sem virar vários nós.                                                |
| <b>Formar o menu por <math>T_{base}</math> e pelas tools do bundle</b> | Um bundle vazio pode usar operações gerais sem exigir uma skill artificial.                                            |
| <b>Capturar observações no runtime</b>                                 | Cada retorno de tool executável vira evidência ordenada; a decisão do modelo não reproduz identificadores do provedor. |
| <b>Associar revisão ao conjunto completo de evidências</b>             | Uma revisão localizada recebe todas as observações do nó e preserva os resultados já concluídos.                       |

Fonte: elaboração própria (2026).

### 1.4 Escopo e modelo de confiança

A MOSAIC 0.2 é uma especificação técnica, não uma implementação de produção. A especificação assume:

- catálogo de skills confiável;
- registro de tools confiável;
- execução em modo YOLO, sem uma fronteira de autorização definida pelo artigo;
- modelo capaz de seguir instruções, produzir saídas estruturadas e chamar tools;
- estado suficiente para preservar resultados e observações entre objetivos.

Não fazem parte do núcleo: defesas contra prompt injection, sandbox, least privilege, confirmação de efeitos, política de credenciais, retries, idempotência, compensação de ações, execução distribuída, scripts e recursos auxiliares de Agent Skills, treinamento de retrievers, implementação de código e avaliação experimental. Esses itens são relevantes em outros recortes, mas não são necessários para especificar a composição many-to-many proposta aqui.

## 1.5 Status normativo e versão do perfil

Este documento define a **MOSAIC 0.2**. Os verbos *deve* e *não deve* indicam requisitos de conformidade; *pode* indica uma opção permitida; exemplos e justificativas são informativos. A especificação não prescreve linguagem, biblioteca, provedor de modelo ou protocolo de tool calling. Ela prescreve os objetos, a autoria de cada objeto e os invariantes necessários para comparar implementações.

## 2 TRABALHOS RELACIONADOS

### 2.1 Agent Skills

A especificação Agent Skills define uma skill como um diretório que contém ao menos um arquivo `SKILL.md`, formado por frontmatter YAML e body em Markdown. `name` e `description` são obrigatórios; `allowed-tools` é opcional e experimental. A especificação também prevê `scripts/`, `references/` e `assets/`, carregados por divulgação progressiva quando necessários (AGENT SKILLS, 2026). O MOSAIC adota o arquivo `SKILL.md` como fonte canônica, mas define um perfil reduzido. O núcleo lê `name`, `description`, `allowed-tools` e o body. Os diretórios opcionais continuam válidos no ecossistema Agent Skills, porém não são necessários para a arquitetura descrita neste artigo e, por isso, ficam fora do perfil central.

Essa delimitação evita afirmar compatibilidade operacional com recursos que o executor mínimo não carrega. O objetivo é compatibilidade no nível da representação de instruções, não uma implementação completa de todos os recursos do padrão.

### 2.2 SkillRouter e SkillRet

O SkillRouter investiga roteamento em catálogos grandes e mostra que o body da skill contém sinal decisivo para distinguir itens funcionalmente próximos. Sua arquitetura usa recuperação de alta cobertura seguida de reranking que lê o conteúdo integral da skill (ZHENG et al., 2026). O SkillRet amplia o problema de retrieval para uma biblioteca realista de milhares de skills e reforça que selecionar instruções relevantes em catálogos grandes é uma tarefa própria, distinta do uso de tools (CHO; KANG; KIM, 2026). O MOSAIC incorpora uma exigência arquitetural desses trabalhos: o segundo estágio de seleção deve ser body-aware. A proposta não exige um retriever específico. Busca lexical, embeddings, modelos treinados ou combinações híbridas são alternativas de implementação. O requisito conceitual é que o roteamento final não dependa apenas do nome e da descrição.

## 2.3 SkillWeaver

O SkillWeaver formaliza a composição de skills por meio de decomposição, retrieval e construção de um DAG. Sua Skill-Aware Decomposition devolve ao planejador sinais do catálogo para corrigir a decomposição inicial (GAO, 2026). Essa interação entre planejamento e retrieval é adotada pelo MOSAIC. A diferença está na unidade de composição. No SkillWeaver, a decomposição procura subtarefas atômicas associadas a skills individuais. No MOSAIC, o nó representa um resultado intermediário e pode receber um bundle de instruções. O feedback do catálogo pode revelar um resultado ausente ou uma dependência inadequada, mas não deve transformar cada skill candidata em um novo nó.

## 2.4 ReAct e Toolformer

ReAct organiza a resolução de tarefas como uma alternância entre decisão, ação e observação, permitindo que novas informações mudem os próximos passos (YAO et al., 2023). Toolformer explora quando chamadas de APIs acrescentam informação ou capacidade externa ao modelo (SCHICK et al., 2023). Ambos sustentam uma separação útil: o modelo interpreta e decide; a tool executa uma operação com contrato observável. O MOSAIC usa essa separação dentro de cada objetivo. O executor pode produzir o resultado diretamente ou alternar chamadas de tools e observações até que os critérios de conclusão sejam atendidos. Uma chamada de tool não se torna automaticamente um nó do plano.

## 2.5 Evidência recente sobre skills, bundles e menus de tools

SkillsBench trata skills como pacotes de conhecimento procedural e mostra que instruções focadas podem ser mais úteis do que documentação acumulada (LI et al., 2026). SkillMOO trata bundles como objetos passíveis de poda e substituição, reforçando que adicionar instruções indiscriminadamente não é equivalente a melhorar o agente (GONG et al., 2026). ToolMenuBench, por sua vez, transforma a construção do menu visível de tools em um problema de primeira classe (BABU; IYER, 2026).

Esses trabalhos ajudam a posicionar o MOSAIC, mas não são incorporados como novos

**Quadro 2 - Posição da MOSAIC em relação aos trabalhos-base**

| Trabalho     | Unidade principal                  | Relação entre tarefa e skill            | Decisão aproveitada pela MOSAIC                            |
|--------------|------------------------------------|-----------------------------------------|------------------------------------------------------------|
| Agent Skills | arquivo ou diretório de instruções | ativação de uma skill                   | SKILL.md como fonte canônica                               |
| SkillRouter  | consulta de retrieval              | seleção e reranking de skills           | body completo como sinal de roteamento                     |
| SkillWeaver  | subtarefa atômica                  | uma skill por subtarefa                 | feedback de retrieval para a decomposição                  |
| ReAct        | trajetória de ação e observação    | não define catálogo de skills           | revisão após observações                                   |
| MOSAIC 0.2   | objetivo orientado a resultado     | zero, uma ou várias skills por objetivo | composição many-to-many e evidência capturada pelo runtime |

Fonte: elaboração própria (2026).

## 3 MODELO CONCEITUAL

### 3.1 Vocabulário mínimo

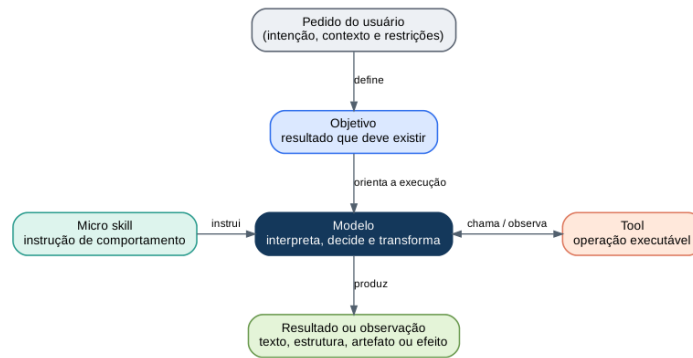
A arquitetura usa os conceitos do Quadro 3.

**Quadro 3 - Vocabulário normativo da MOSAIC 0.2**

| Conceito          | Definição                                                                                                                       | Pergunta respondida                                      |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| Pedido            | Intenção, contexto e restrições fornecidos pelo usuário.                                                                        | O que o usuário quer alcançar?                           |
| Objetivo          | Resultado intermediário que deve existir ao final de uma parte da tarefa.                                                       | O que precisa estar verdadeiro?                          |
| Nó de objetivo    | Objetivo inserido em um plano, com dependências, critérios e status.                                                            | Que unidade pode ser executada agora?                    |
| Skill             | Instrução reutilizável que altera como o modelo executa um trabalho.                                                            | Que comportamento deve orientar o modelo?                |
| Tool              | Operação executável com nome, entrada e retorno observável.                                                                     | Que ação concreta o runtime executa?                     |
| Bundle            | Sequência ordenada de skills ativadas para um objetivo.                                                                         | Que instruções atuam em conjunto e em que ordem?         |
| Decisão do nó     | Saída semântica do modelo: status, critérios, resultado, pedido de revisão e razão. Não contém observações nem IDs do provedor. | O que o modelo concluiu semanticamente?                  |
| Observação        | Registro criado pelo runtime para um retorno de tool executável, com correlação interna, nome, entrada, saída e ordem.          | Que evidência operacional foi realmente retornada?       |
| Resultado do nó   | Objeto materializado pelo runtime ao combinar a decisão do modelo com todas as observações correlacionadas do nó.               | Que decisão e evidências o runtime aceita conjuntamente? |
| Pedido de revisão | Descrição semântica da suposição invalidada e do efeito solicitado, associada automaticamente a todas as observações do nó.     | Por que e com que efeito o plano deve mudar?             |
| Estado projetado  | Subconjunto causal do estado entregue ao objetivo atual.                                                                        | Que resultados anteriores são relevantes para este nó?   |
| Plano             | DAG de nós de objetivo e dependências.                                                                                          | Em que ordem os resultados precisam existir?             |

Fonte: elaboração própria (2026).

**Figura 1** – Separação de papéis entre pedido, objetivo, skill, modelo e tool



Fonte: elaboração própria (2026).

A separação da Figura 1 é normativa para a arquitetura. O objetivo descreve o resultado; a skill orienta o comportamento; a tool executa uma operação; o modelo decide como combinar esses elementos. Nenhum dos conceitos substitui os demais.

### 3.2 O que torna uma skill “micro”

“Micro” não significa uma única linha, uma única chamada ou uma única tool. Uma micro skill é pequena em responsabilidade, não necessariamente em número de passos. Ela deve atender a quatro propriedades:

1. **Coerência:** ensina uma preocupação comportamental identificável, como fundamentar afirmações em evidências ou comparar alternativas sob critérios explícitos.
2. **Reutilização:** pode ser aplicada a mais de um pedido ou objetivo sem depender de um workflow inteiro.
3. **Composição:** pode atuar ao lado de outras skills sem assumir controle exclusivo da tarefa.
4. **Carga integral:** seu body é conciso o suficiente para ser carregado completo quando selecionado.

`evidence-grounded-summary` é micro porque ensina como ligar afirmações a dados. `uncertainty-reporting` é micro porque ensina como explicitar premissas e limitações. `latest-document-reading` também pode ser micro mesmo usando `list` e `read`, pois sua responsabilidade coerente é identificar e disponibilizar o documento correto.

Em contraste, `financial-agent-that-does-everything` mistura descoberta, leitura, análise, decisão, redação e persistência. `use-read-when-reading` apenas repete o nome da tool e não acrescenta comportamento. O primeiro é amplo demais; o segundo é vazio demais.

### 3.3 Granularidade do objetivo

Um objetivo descreve um resultado, não uma sequência de operações. A frase “obter o relatório financeiro mais recente e válido” é um objetivo. A frase “chamar `list`, ordenar datas e chamar

read” é uma trajetória de execução. Uma nova etapa é justificada quando pelo menos uma das condições abaixo ocorre:

- o resultado será consumido por outro objetivo;
- uma observação externa pode mudar o restante do plano;
- uma parte possui critério de conclusão próprio;
- duas partes podem ser executadas independentemente;
- o usuário pediu resultados ou artefatos separados.

A existência de uma tool ou de uma skill não é, sozinha, motivo para criar um nó. Listar arquivos, comparar nomes e ler um documento podem ocorrer dentro do mesmo objetivo. Da mesma forma, aplicar duas skills cognitivas ao mesmo conjunto de dados não exige dois objetivos.

### 3.4 Relações many-to-many

Considere um plano  $P = (G, E)$ , em que  $G$  é o conjunto de objetivos e  $E$  contém dependências acíclicas. Para cada objetivo  $g$ , o seletor produz um bundle ordenado:

$$B(g) = \langle s_1, \dots, s_k \rangle, \quad 0 \leq k \leq K_{\max}. \quad (1)$$

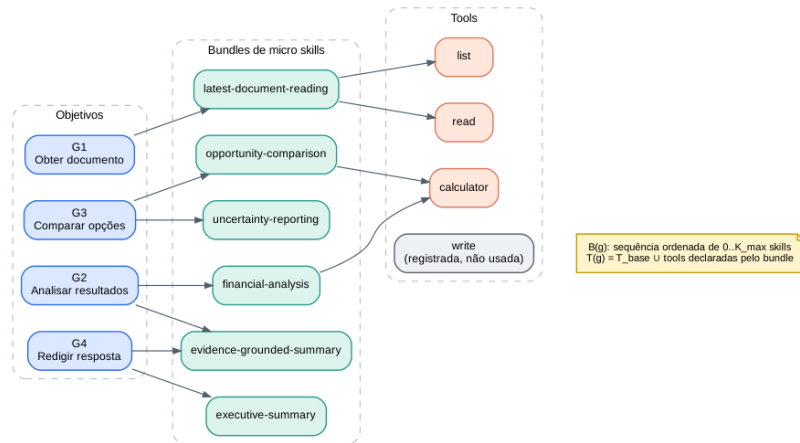
$K_{\max}$  é um limite pequeno configurado pelo runtime. Ele não define quantas skills existem no catálogo; apenas impede que um único objetivo acumule instruções demais. Cada skill  $s$  pode declarar um conjunto  $A(s)$  de tools em `allowed-tools`. O menu visível para o objetivo é:

$$T(g) = T_{\text{base}} \cup \bigcup_{s \in B(g)} A(s). \quad (2)$$

$T_{\text{base}}$  é o conjunto de tools gerais que o runtime decide expor a todos os objetivos. No perfil YOLO máximo, pode-se definir  $T_{\text{base}} = T_{\text{registry}}$ , tornando todas as tools registradas sempre visíveis. Em um perfil reduzido, `calculator` pode ser base enquanto integrações específicas, como `slack_send_message`, entram pelo bundle. A fórmula resolve um caso importante: se  $B(g) = \emptyset$ , o objetivo ainda pode usar  $T_{\text{base}}$ . Portanto, uma operação geral não exige uma skill artificial. Se nenhuma tool for necessária, o modelo conclui o objetivo cognitivamente.



**Figura 2** – Relações many-to-many entre objetivos, micro skills e tools



Fonte: elaboração própria (2026).

A Figura 2 também mostra que uma skill pode orientar vários objetivos, que um objetivo pode combinar várias skills e que a mesma tool pode aparecer sob procedimentos diferentes. A semântica da operação permanece estável; o comportamento muda conforme o bundle.

### 3.5 Semântica do bundle ordenado

O bundle é uma sequência, não um conjunto sem ordem. O executor injeta bodies no contexto em uma ordem observável, e essa ordem pode afetar o modelo. Para evitar que duas implementações igualmente conformes usem convenções incompatíveis, o MOSAIC 0.2 define a regra mínima abaixo:

- o reranker produz uma ordem de aplicabilidade;
- o seletor percorre essa ordem e retém até Kmax skills que acrescentem comportamentos distintos e necessários;
- a ordem final do bundle preserva a posição do reranking entre as skills selecionadas;
- empates de score são resolvidos pelo nome canônico da skill em ordem lexicográfica crescente;
- skills redundantes, conflitantes ou fora do objetivo são omitidas;
- o seletor pode retornar uma sequência vazia.

Na MOSAIC 0.2, a ordem de aplicabilidade é deliberadamente usada como ordem de composição. Essa é uma convenção de referência, não uma afirmação de que ela seja ótima. Uma extensão pode adotar precedência mais rica, desde que declare outra versão. Os bodies devem ser delimitados pelo nome canônico da skill para impedir fusão acidental entre instruções adjacentes.

## 4 ARQUITETURA MÍNIMA

### 4.1 Visão geral

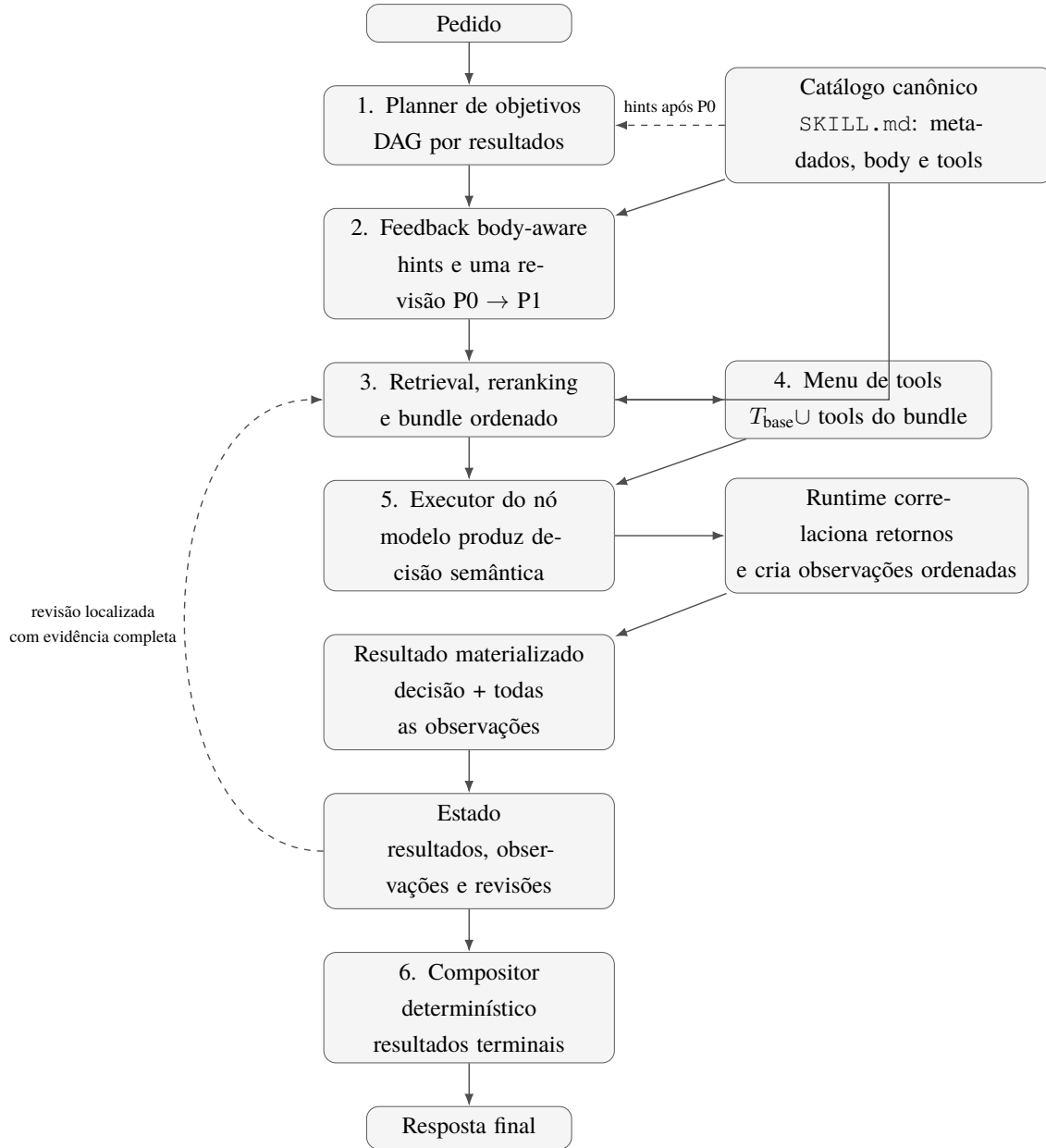
A MOSAIC 0.2 possui seis estágios de runtime, alimentados por um catálogo canônico e conectados por um estado compartilhado:

1. planejador de objetivos; 2. feedback do catálogo e revisão única do plano inicial; 3. roteador de skills e seletor de bundle; 4. montagem do menu de tools; 5. executor do nó; 6. compositor final determinístico.

O estado preserva resultados e observações criadas pelo runtime. Para cada nó, o modelo produz apenas uma decisão semântica. O runtime correlaciona cada chamada executável ao retorno correspondente, cria observações na ordem dos resultados e materializa o resultado do nó anexando todas elas à decisão. Quando uma dessas evidências invalida uma suposição estrutural, o planejador revisa somente a parte não concluída do plano. O compositor final

não realiza uma segunda síntese: ele apenas entrega os resultados que já foram produzidos por objetivos terminais.

**Figura 3 - Arquitetura mínima da MOSAIC 0.2**



Fonte: elaboração própria (2026).

A Figura 3 separa duas responsabilidades. O catálogo prepara representações para busca; o runtime transforma um pedido em resultados. O catálogo não define a granularidade do plano, e o planner não escolhe tools diretamente.

## 4.2 Catálogo canônico e índice body-aware

Cada skill é representada por um `SKILL.md`. O núcleo mantém somente os dados necessários para recuperação e execução:

- name;
- description;
- body completo;
- lista normalizada de `allowed-tools`;
- texto de indexação derivado deterministicamente desses campos.

O carregamento do catálogo é *fail-fast* para os campos normativos da MOSAIC 0.2. A implementação deve rejeitar o catálogo quando houver nome de skill duplicado, campo obrigatório ausente, body vazio ou nome de tool declarado que não exista em Tregistry. Tools repetidas em uma mesma skill são deduplicadas. Campos desconhecidos podem ser preservados, mas não alteram o núcleo. Uma versão anterior da arquitetura previa um compilador baseado em modelo para inferir `useWhen`, `behaviors`, `outputs` e outros campos. Esse componente foi removido do núcleo. A recuperação `body-aware` já pode operar sobre `name`, `description` e `body`; o `reranker` lê a fonte original. Metadados inferidos podem ser úteis em implementações específicas, mas não são necessários para expressar a composição. O índice pode ser lexical, vetorial ou híbrido. O MOSAIC não fixa o mecanismo. A única exigência é preservar um caminho do resultado de retrieval até o body canônico, pois o reranking e a execução dependem das instruções reais, não apenas de resumos derivados.

### 4.3 Planejamento inicial por resultados

O planejador recebe o pedido e produz um DAG inicial P0. Nessa primeira passagem, ele não recebe nomes de skills. Isso reduz a tendência de copiar a estrutura física do catálogo. Cada nó deve possuir:

- identificador único no plano; • objetivo orientado a resultado; • dependências que referenciam apenas IDs existentes; • critérios `doneWhen`; • status inicial `pending`.

Para o pedido financeiro, um plano inicial plausível é:

1. obter o relatório mais recente e válido; 2. interpretar os resultados relevantes; 3. comparar oportunidades; 4. redigir a resposta.

O planner não precisa antecipar chamadas como `list`, `read` ou `calculator`. Essas decisões pertencem ao executor de cada nó.

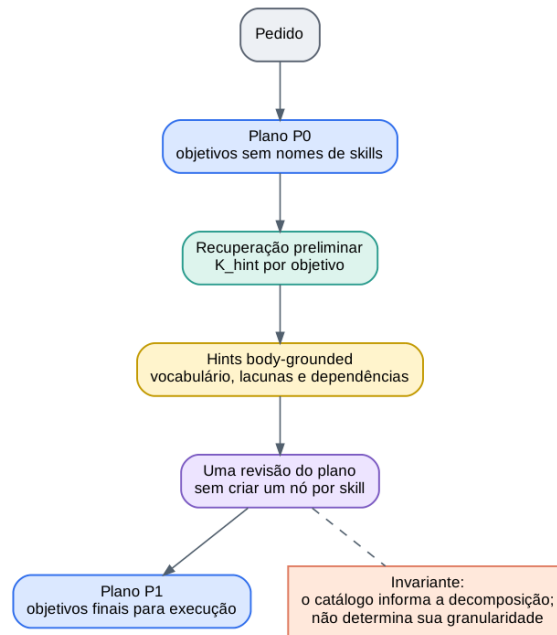
### 4.4 Feedback do catálogo e revisão única

Depois de P0, o runtime executa uma recuperação preliminar por objetivo. Para cada consulta, no máximo `Khint` candidatas são convertidas em hints curtas, fundamentadas no body. As hints podem revelar:

- vocabulário do domínio que não apareceu no pedido; • um resultado intermediário ausente; • uma divisão artificial; • uma dependência inadequada.

O planner recebe essas hints e pode produzir P1. A revisão ocorre uma vez antes da execução. Depois dela, o roteamento é refeito para os objetivos finais. Esse fluxo evita revisar um plano antes de gerar os sinais do catálogo.

**Figura 4** – Feedback do catálogo sobre a decomposição inicial



Fonte: elaboração própria (2026).

A regra de controle é simples: uma skill pode revelar que “analisar” e “resumir” são resultados semanticamente diferentes, mas sua mera existência não cria um nó. O catálogo informa o planner; não governa sua granularidade.

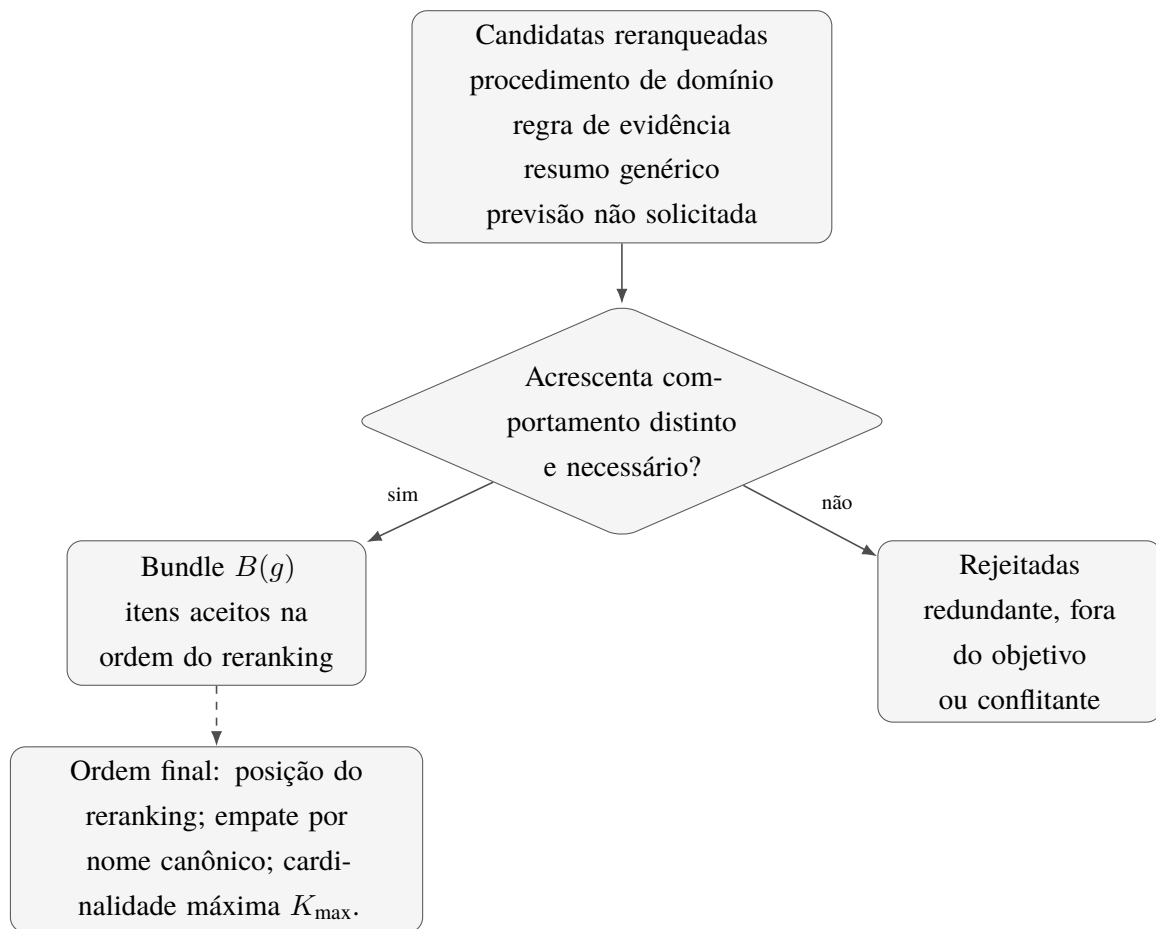
## 4.5 Retrieval, reranking e seleção do bundle

Para cada objetivo pronto, o roteador constrói uma consulta com:

- descrição do objetivo;
- critérios `doneWhen`;
- termos relevantes do pedido original;
- saídas necessárias dos objetivos anteriores.

O primeiro estágio privilegia cobertura e retorna até  $K_{\text{retrieve}}$  candidatas. O segundo compara as candidatas usando o body completo. Em seguida, o seletor percorre a lista reranqueada e retém, até  $K_{\text{max}}$ , apenas as skills que acrescentam comportamentos distintos e necessários.

**Figura 5 - Semântica mínima da seleção de bundle**



Fonte: elaboração própria (2026).

A seleção é baseada no comportamento ensinado pela skill. Uma skill não deve ser escolhida apenas porque declara uma tool útil. Se uma operação geral já estiver em Tbase, ela continua disponível mesmo quando nenhuma skill é selecionada.

## 4.6 Montagem do menu de tools

Após a seleção, o runtime calcula  $T(g)$  pela fórmula apresentada anteriormente. Não existe um tool router independente no núcleo. Essa remoção é deliberada: um segundo roteador duplicaria a decisão de relevância antes que a hipótese básica fosse demonstrada.

**Quadro 4 - Exemplos de montagem do menu de tools**

| Configuração                                            | Bundle                                                               | Menu resultante                                   |
|---------------------------------------------------------|----------------------------------------------------------------------|---------------------------------------------------|
| $T_{\text{base}} = \{\text{calculator}\}$               | vazio                                                                | $\{\text{calculator}\}$                           |
| $T_{\text{base}} = \{\text{calculator}\}$               | latest-document-reading<br>declara list e read                       | $\{\text{calculator}, \text{list}, \text{read}\}$ |
| $T_{\text{base}} = \{\text{calculator}\}$               | slack-message declara<br>slack_list_channels e<br>slack_send_message | união das três tools                              |
| YOLO máximo:<br>$T_{\text{base}} = T_{\text{registry}}$ | qualquer                                                             | todas as tools registradas                        |

Fonte: elaboração própria (2026).

`allowed-tools` é, neste perfil, um vínculo declarativo entre skill e menu de execução. Não é tratado como fronteira de autorização ou política de segurança.

## 4.7 Montagem do contexto e projeção de estado

O executor recebe somente o contexto necessário para o objetivo atual. A montagem deve seguir a ordem estável abaixo:

1. instruções do runtime; 2. pedido original e restrições globais; 3. objetivo atual e seus critérios `doneWhen`; 4. estado projetado; 5. bodies das skills do bundle, cada um entre delimitadores que contenham o nome canônico; 6. schemas das tools de  $T(g)$ .

Por padrão, o estado projetado de um objetivo contém:

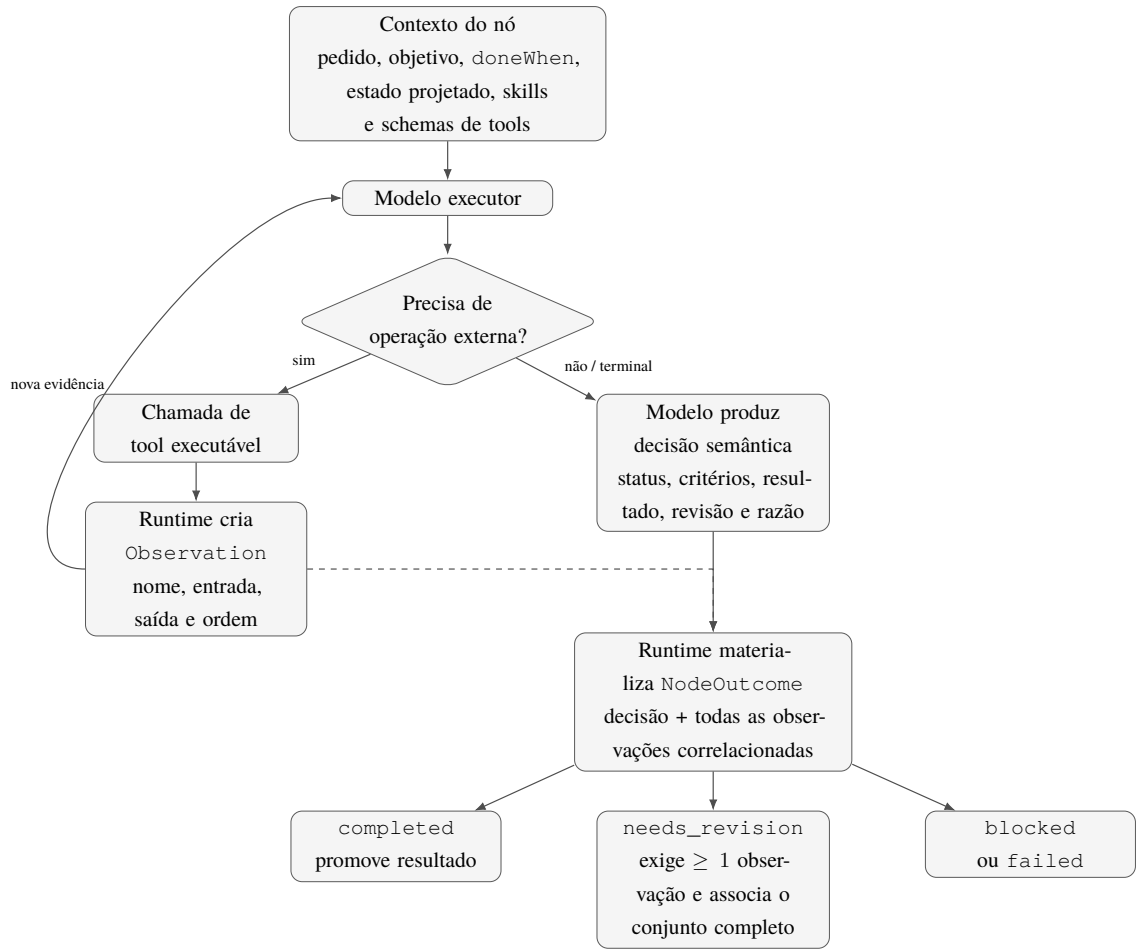
- saídas dos ancestrais transitivos do nó no DAG;
- restrições globais do pedido;
- referências para artefatos necessários à execução.

Resultados de ramos sem dependência causal com o objetivo atual são omitidos, salvo quando o plano os referencia explicitamente. Essa regra reduz contexto e torna a projeção audível sem introduzir um novo subsistema de memória.

## 4.8 Executor do nó

O modelo pode concluir diretamente ou chamar tools. Cada retorno bem-sucedido de tool executável volta ao mesmo nó e é convertido pelo runtime em uma `Observation`; a tool reservada para a saída estruturada terminal nunca é uma observação. A decisão final do modelo contém somente status, avaliações ordenadas de `doneWhen`, resultado, pedido semântico de revisão e razão. O runtime rejeita dados de chamada ou retorno ausentes, duplicados ou não correlacionados e então materializa o `NodeOutcome` ao combinar a decisão com todas as observações, na ordem dos resultados. A evidência escrita em cada avaliação de critério continua sendo prosa do modelo; a associação das observações ocorre no nível do nó.  $L_{\text{node}}$  limita o número de turnos. Se os critérios não foram satisfeitos e não há ação útil, o nó termina como `blocked`; saídas inválidas ou falhas terminais podem produzir `failed`.

**Figura 6 - Loop de execução e materialização do resultado do nó**



Fonte: elaboração própria (2026).

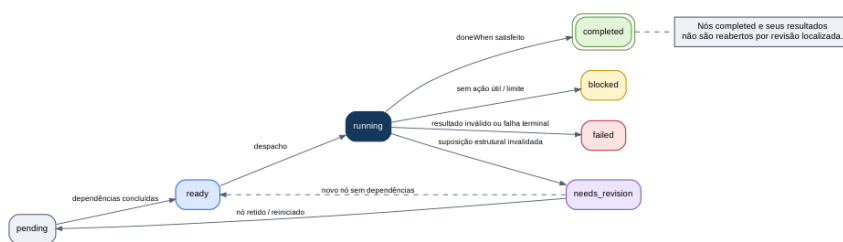
## 4.9 Ciclo de vida do nó e revisão localizada

O status de um nó pode ser `pending`, `ready`, `running`, `completed`, `needs_revision`, `blocked` ou `failed`. Um nó `pending` torna-se `ready` quando todas as dependências estão `completed`. O scheduler o coloca em `running`. A satisfação de `doneWhen` leva a `completed`. Uma

decisão `needs_revision` descreve a suposição estrutural invalidada e o efeito solicitado. Esse status exige ao menos uma observação correlacionada. O runtime associa automaticamente o pedido de revisão a todas as observações produzidas pelo nó, ainda que o planner conclua depois que algumas são irrelevantes. A ausência de ação útil ou o esgotamento de  $L_{\text{node}}$  produz `blocked`; uma falha terminal produz `failed`.



**Figura 7** – Máquina de estados mínima de um nó



Fonte: elaboração própria (2026).

Quando ocorre revisão, o planner recebe nome da tool, entrada e saída de cada observação em ordem, mas não recebe `callId` nem qualquer identificador opaco do provedor. Ele decide a relevância a partir da suposição invalidada e do efeito solicitado:

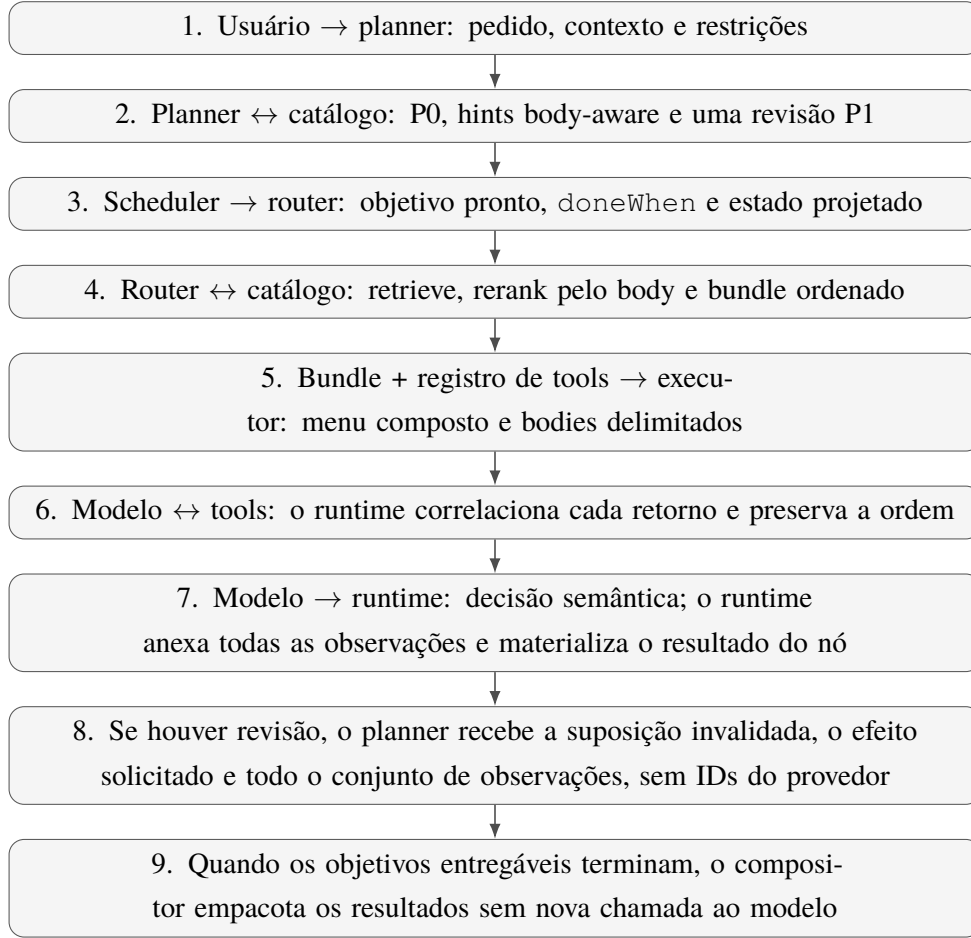
- objetivos `completed` permanecem concluídos e seus resultados permanecem no estado;
- somente o nó atual ainda incompleto e objetivos não iniciados podem ser retidos, substituídos, removidos ou inseridos;
- IDs de nós sobreviventes permanecem estáveis;
- dependentes de um nó removido devem ser removidos ou reconectados explicitamente;
- o plano revisado deve continuar acíclico e referenciar apenas IDs existentes;
- o nó atual pode ser mantido com o mesmo ID e reiniciado como `pending`, ou substituído quando a nova decomposição exigir outra unidade de resultado.

`Rmax` limita o número de revisões localizadas em uma execução. O artigo não prescreve um valor universal; o runtime deve declarar o valor usado. Essa guarda impede ciclos infinitos sem transformar o núcleo em um protocolo completo de recuperação.

## 4.10 Composição da resposta final

Objetivos terminais produzem o conteúdo semanticamente final. Quando a tarefa exige síntese, redação ou decisão, o plano deve conter um objetivo terminal responsável por essa transformação, como `G4` no exemplo financeiro. A função `compor_resposta` não chama o modelo e não aplica skills novamente. Ela seleciona resultados de nós terminais marcados para entrega, ordena-os de forma estável pela ordem topológica e empacota texto, referências de artefatos e efeitos observáveis. Assim, a resposta é produzida exatamente uma vez e o trace identifica qual objetivo realizou a síntese.

**Figura 8 - Sequência de uma execução completa da MOSAIC 0.2**



Fonte: elaboração própria (2026).

## 5 ALGORITMO CENTRAL

A arquitetura pode ser expressa pelo algoritmo abstrato abaixo. Ele especifica a ordem das decisões sem fixar linguagem, biblioteca, modelo ou protocolo de erros.

### Algoritmo 1 - Execução da MOSAIC 0.2

entrada: pedido  $q$ , catálogo  $S$ , registro  $T\_registry$ , base  $T\_base$ ,  
 $K\_retrieve$ ,  $K\_hint$ ,  $K\_max$ ,  $L\_node$  e  $R\_max$

```

validar_catalogo_e_registro( $S$ ,  $T\_registry$ ,  $T\_base$ )
 $P0 \leftarrow$  planejar_objetivos( $q$ )           // sem nomes de skills
 $H \leftarrow$  recuperar_hints_body_aware( $P0$ ,  $S$ ,  $K\_hint$ )
 $P \leftarrow$  revisar_uma_vez( $q$ ,  $P0$ ,  $H$ )
validar_dag( $P$ )
estado  $\leftarrow$  vazio
revisoes  $\leftarrow 0$ 
  
```

```

enquanto houver objetivo não terminal em P:
    wave <- objetivos prontos em ordem determinística

para cada g em wave:
    C <- recuperar_e_reranquear(g, q, projetar_estado(g, P, estado),
                                S, K_retrieve)
    B <- selecionar_bundle_ordenado(C, K_max)
    T <- T_base união tools_declaradas(B)
    contexto <- montar_contexto(q, g, projetar_estado(g, P, estado),
                                ↪ B, T)

    historico, decisao <- executar_ate_decisao(contexto, L_node)
    observacoes <- correlacionar_retornos_executaveis(historico)
    // falhar se chamada/retorno estiver ausente, duplicado ou não
    ↪ correlacionado
    // excluir a tool terminal usada somente para saída estruturada
    outcome <- materializar(decisao, observacoes)

anexar observacoes ao estado na ordem da wave e dos retornos de
    ↪ cada nó

para cada outcome da wave:
    se outcome.status = completed:
        promover(outcome.resultado)
    senão se outcome.status = needs_revision:
        exigir tamanho(outcome.observacoes) >= 1
        associar(outcome.revisionRequest, todas outcome.observacoes)
        exigir revisoes < R_max
        P <- revisar_objetivo_atual_e_restantes(
            P, outcome.revisionRequest,
            evidencias_sem_callId(outcome.observacoes))
        validar_dag(P)
        revisoes <- revisoes + 1
    senão:
        registrar outcome como blocked ou failed

retornar compor_resposta_deterministicamente(q, P, estado)

```

O algoritmo mantém sete invariantes de conformidade:

1. objetivos descrevem resultados, não chamadas de tools nem nomes de skills;
2. o planner inicial não depende dos nomes das skills;
3. o bundle pode conter zero, uma ou várias skills e possui ordem observável;
4. tools gerais podem existir fora das skills por meio de  $T_{base}$ ;
5. o body

canônico é usado tanto no reranking quanto na execução; 6. toda observação executável é criada, correlacionada e ordenada pelo runtime, nunca pelo modelo; 7. uma revisão recebe o conjunto completo de observações, preserva nós concluídos e altera apenas o nó atual incompleto e a parte ainda não iniciada do plano.

Uma implementação que preserve esses invariantes e os contratos do Apêndice A pode variar livremente em modelo, índice, linguagem, formato de serialização e protocolo de tool calling.

## 6 PERFIL MOSAIC 0.2 PARA SKILL.MD

### 6.1 Subconjunto suportado

O perfil mínimo usa o subconjunto do Quadro 5.

**Quadro 5 - Elementos de SKILL.md na MOSAIC 0.2**

| Elemento                                 | Tratamento na MOSAIC 0.2                                       |
|------------------------------------------|----------------------------------------------------------------|
| name                                     | obrigatório; identificador único da skill                      |
| description                              | obrigatória; sinal inicial de retrieval                        |
| allowed-tools                            | opcional; normalizado para lista de nomes de tools             |
| body Markdown                            | obrigatório; fonte principal de roteamento e instrução         |
| license,<br>compatibility, meta-<br>data | podem ser preservados, mas não alteram os contratos normativos |
| scripts/, references/,<br>assets/        | fora da especificação mínima                                   |

Fonte: elaboração própria (2026).

### 6.2 Gramática e normalização de allowed-tools

A forma canônica do MOSAIC é uma sequência YAML:

```
allowed-tools:  
  
- calculator  
- read
```

Para compatibilidade de ingestão, uma implementação pode aceitar um escalar com nomes separados por espaços, como `allowed-tools: list read`. O parser deve normalizar as duas formas para `string[]`, remover duplicatas e preservar a ordem de primeira ocorrência. Nomes de tools são identificadores exatos, sensíveis a caixa quando o registro também o for. Uma referência que não resolva em Tregistry invalida o carregamento do catálogo.

## 6.3 Estrutura recomendada do body

O formato do body permanece livre, mas uma micro skill legível tende a responder cinco perguntas:

1. qual comportamento ela ensina; 2. quando deve ser usada; 3. quando não deve ser usada; 4. qual procedimento ou critério deve ser seguido; 5. como reconhecer a conclusão.

Uma estrutura recomendada é:

```
# Nome legível

## Purpose
Explique a preocupação comportamental da skill.

## Use this skill when
- descreva situações aplicáveis.

## Do not use this skill when
- descreva situações não aplicáveis.

## Procedure
1. descreva o método ou os critérios.

## Completion
Descreva o resultado esperado.
```

Os títulos não são obrigatórios. O requisito é semântico: o body precisa acrescentar comportamento que o nome da tool ou uma descrição genérica não expressam.

## 6.4 Exemplo cognitivo com tool opcional

```
---
name: financial-analysis
description: Interprets reported financial results, compares periods,
and identifies material business changes. Use when financial evidence
must be understood rather than merely copied.
allowed-tools:

- calculator
---

# Financial Analysis
Compare period, units, revenue, margins, cash generation and material
```

costs.  
Separate recurring changes from one-time events.  
Connect claims to reported figures and state missing information.  
Use calculator only when a derived metric is necessary.  
Completion: the main changes, their evidence and the principal  
limitation  
are explicit.

A skill continua cognitiva: seu principal valor é o método de análise. calculator apenas acrescenta uma operação possível.

## 6.5 Exemplo de uma skill com várias tools

```
---
name: latest-document-reading

description: Finds and reads the most relevant recent document. Use
  ↳ when

the user asks for the latest report, memo, statement or specification.
allowed-tools:

- list
- read
---
```

# Latest Document Reading  
Identify the requested document type and period.  
List plausible candidates, compare dates and status markers, then  
↳ read

enough  
content to confirm the correct document before returning its content  
reference.  
Completion: the selected document matches the requested type and  
↳ period,  
and the required content is available to the next objective.

A skill orienta várias chamadas possíveis sem criar vários nós. list, leituras parciais e leitura final pertencem ao mesmo resultado: disponibilizar o documento correto.

## 6.6 Semântica de `allowed-tools`

No MOSAIC, `allowed-tools` significa:

*Quando a skill está ativa, os nomes declarados são adicionados ao menu de tools do objetivo.*

A lista não descreve o propósito da skill, não substitui o body e não deve ser o único critério de seleção. Duas skills podem declarar `read` e ensinar comportamentos diferentes. Uma tool pode estar em Tbase e também aparecer em `allowed-tools`; a união remove duplicatas. Essa semântica é funcional, não securitária. O artigo assume que o catálogo e o registro são confiáveis e não define permissões, confirmação ou isolamento.

## 7 EXEMPLOS DE COMPOSIÇÃO

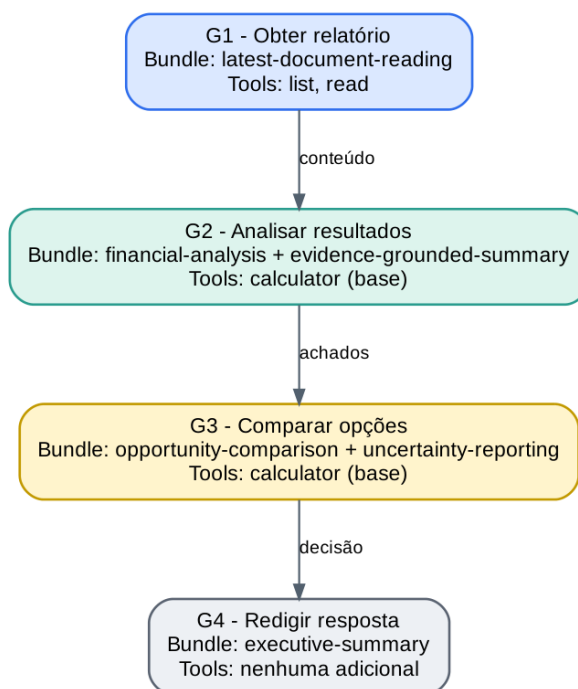
### 7.1 Exemplo financeiro de ponta a ponta

Pedido:

*Leia o último relatório financeiro, interprete os resultados e retorne a melhor oportunidade.*

Considere  $T_{base} = \{\text{calculator}\}$ . O plano possui quatro objetivos.

Figura 9 – Plano, bundles e tools do exemplo financeiro



Fonte: elaboração própria (2026).

No primeiro objetivo, `latest-document-reading` acrescenta `list` e `read`. O executor pode listar arquivos, ler cabeçalhos e confirmar o documento sem transformar cada chamada em um nó. No segundo objetivo, duas skills orientam a mesma análise: uma fornece o método financeiro; outra exige que os achados permaneçam ligados às evidências. `calculator` está disponível por ser base, mas seu uso não é obrigatório. O terceiro objetivo usa duas skills cognitivas para gerar e comparar alternativas sob incerteza. O quarto organiza a comunicação e produz o conteúdo final. Depois de G4, o compositor apenas empacota o resultado de G4; não executa nova síntese.

## 7.2 Bundle vazio com tool base

Pedido:

*Leia o valor bruto da fixture invoice-028 e calcule 17,5% desse valor.*

Considere  $T_{\text{base}} = \{\text{read\_fixture}, \text{calculator}\}$ . O plano pode conter um único objetivo: “produzir o valor derivado correto”. O seletor pode retornar  $B(g) = \emptyset$ , porque ler um identificador conhecido e executar uma operação aritmética não exige um procedimento especializado. O menu permanece capaz de acessar um valor externo e calcular o resultado. Esse exemplo demonstra duas propriedades: skills não são pedágios obrigatórios para qualquer ação, e um caso de validação pode tornar a tool causalmente necessária ao esconder o valor da fixture do prompt. Um cálculo cujo operando já aparece no pedido ilustra apenas disponibilidade, não necessidade empírica de  $T_{\text{base}}$ .

## 7.3 A mesma tool sob skills diferentes

Quadro 6 - Reuso da mesma tool sob procedimentos diferentes

| Objetivo                           | Skill                                | Comportamento acrescentado                               |
|------------------------------------|--------------------------------------|----------------------------------------------------------|
| identificar obrigações contratuais | <code>contract-analysis</code>       | localizar partes, obrigações, prazos e exceções          |
| explicar fluxo de autenticação     | <code>source-code-explanation</code> | seguir módulos, chamadas e dependências                  |
| interpretar resultados financeiros | <code>financial-analysis</code>      | identificar período, unidades, variações e materialidade |

Fonte: elaboração própria (2026).

A operação é a mesma: disponibilizar conteúdo. A interpretação muda porque o bundle muda.

## 7.4 Uma skill, várias tools e várias chamadas

Pedido:

*Encontre o relatório do segundo trimestre e leia as seções de receita e caixa.*

Um único objetivo pode executar a trajetória:



```
list candidatos
-> read cabeçalho do arquivo A
-> read cabeçalho do arquivo B
-> selecionar o arquivo B
-> read seção de receita
-> read seção de caixa
-> concluir com uma referência ao conteúdo
```

O número de chamadas não determina o número de objetivos. O resultado intermediário relevante é “o conteúdo correto está disponível”.

## 7.5 Produção e persistência de um artefato

Pedido:

*Escreva um README de instalação e salve em README.md.*

O plano pode separar:

1. produzir o conteúdo do README; 2. persistir o conteúdo em README.md.

A separação não ocorre porque existe a tool `write`. Ela ocorre porque há dois resultados observáveis: conteúdo pronto e arquivo persistido. O primeiro objetivo pode usar `technical-document-writing` sem tools. O segundo pode usar `file-writing`, que declara `write`. Se o usuário pedisse apenas “escreva um README”, o segundo objetivo não existiria; a resposta seria entregue no chat.

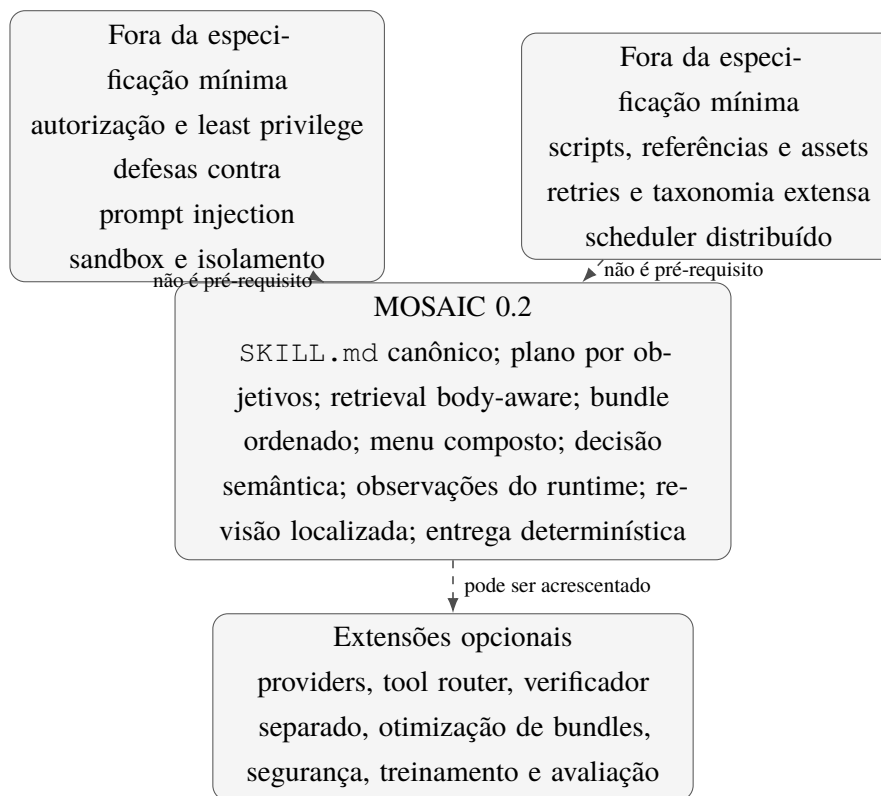
## 8 DECISÕES DE REDUÇÃO E NÃO OBJETIVOS

**Quadro 7 - Elementos removidos da especificação mínima**

| Elemento removido                                                                     | Razão                                                                                                |
|---------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| Compilador por LLM com muitos campos inferidos                                        | <code>name</code> , <code>description</code> e <code>body</code> já sustentam indexação e reranking. |
| Tool router independente                                                              | $T_{base}$ e a união de <code>allowed-tools</code> resolvem o menu mínimo.                           |
| Camada de capabilities e providers                                                    | Nomes diretos de tools bastam enquanto não houver implementações equivalentes.                       |
| Resolvedor de <code>scripts/</code> , <code>references/</code> e <code>assets/</code> | O conceito é expresso carregando apenas <code>SKILL.md</code> .                                      |
| Otimizador formal de bundles                                                          | Seleção ordenada, limite e critérios qualitativos são suficientes.                                   |
| Linguagem formal de precedência                                                       | A ordem estável do reranking é a convenção da MOSAIC 0.2.                                            |
| Verificador separado                                                                  | <code>doneWhen</code> pode ser avaliado pelo executor na especificação mínima.                       |
| Protocolo completo de erros                                                           | Status de nó, limites e pedido semântico de revisão descrevem o fluxo central.                       |
| IDs do provedor em saídas do modelo                                                   | Correlação pertence ao runtime; o modelo não deve copiar referências opacas.                         |
| Segurança, autorização e prompt injection                                             | A especificação assume catálogo, tools e runtime confiáveis em modo YOLO.                            |
| Scheduler distribuído e paralelismo formal                                            | Um DAG expressa a ordem necessária sem infraestrutura adicional.                                     |

Fonte: elaboração própria (2026).

**Figura 10 - Fronteira entre a MOSAIC 0.2 e extensões operacionais**



Fonte: elaboração própria (2026).

A fronteira da Figura 10 não impede extensões. Ela estabelece uma ordem: primeiro, tornar a composição conceitualmente clara e implementável; depois, acrescentar mecanismos que casos reais justifiquem. Uma implementação endurecida pode envolver o MOSAIC com política, sandbox, filtragem de tools e observabilidade sem alterar a relação fundamental entre objetivo, bundle, menu e executor.

## 9 DISCUSSÃO E LIMITAÇÕES

### 9.1 Ausência de validação empírica

O artigo apresenta uma especificação técnica. Não demonstra que bundles superam top-1, que o feedback do catálogo melhora planos ou que a fórmula de tools produz maior sucesso. Essas são hipóteses decorrentes do desenho e exigem implementação e avaliação separadas. As formulações do texto devem ser lidas como propriedades da arquitetura, não como resultados experimentais.

## 9.2 Dependência do modelo

Planner, reranker, seletor e executor dependem da capacidade do modelo de seguir instruções e distinguir comportamentos próximos. Um modelo fraco pode produzir objetivos inadequados, incluir skills redundantes ou ignorar partes do bundle. O MOSAIC organiza o problema, mas não elimina a dependência da competência do modelo.

## 9.3 Qualidade e tamanho dos bodies

Retrieval e execução dependem da qualidade do `SKILL.md`. Um body vago produz sinal vago. Como o núcleo carrega bodies completos, skills excessivamente longas aumentam custo e podem diluir atenção. A resposta mínima é disciplinar a autoria e manter Kmax pequeno. Compressão automática e carregamento seletivo são extensões, não componentes centrais.

## 9.4 Conflitos e ordem

O seletor evita conflitos de forma qualitativa e preserva a ordem do reranking, com desempate determinístico. Isso torna a implementação comparável, mas não garante que a ordem seja ótima quando instruções possuem tensão implícita. Uma linguagem formal de precedência pode ser necessária em catálogos maiores, porém pertence a um perfil posterior se evidência empírica justificar o custo.

## 9.5 Menu simples de tools

Tbase pode expor tools que o objetivo não usa. No perfil YOLO máximo, todas as tools ficam visíveis. Essa escolha prioriza simplicidade conceitual e resolve o caso de bundle vazio com tool. Filtragem por estado, custo, causalidade ou risco pertence a arquiteturas de menu mais sofisticadas.

## 9.6 Subconjunto de Agent Skills

O perfil não executa scripts nem carrega referências e assets. Skills que dependem desses recursos não são integralmente executáveis pelo núcleo. Essa restrição é explícita e evita que o artigo prometa uma camada de resolução que não participa da ideia central.

## 9.7 Revisão localizada

O pedido de revisão pressupõe que o executor reconheça quando uma evidência altera o plano, e que o planner preserve resultados concluídos. Capturar automaticamente todas as observações elimina seleção ou cópia incorreta de referências pelo modelo, mas não garante que ele formule corretamente a suposição invalidada nem que o planner atribua relevância às evidências certas.

## 9.8 Contratos mínimos não substituem operação de produção

Os contratos do perfil resolvem ambiguidade entre implementações, mas não especificam persistência distribuída, consistência transacional, retries, idempotência, observabilidade com-pleta ou recuperação de infraestrutura. Eles definem o que deve ser trocado entre os componentes, não como cada componente deve ser operacionalizado.

## 10 CONSIDERAÇÕES FINAIS

A MOSAIC 0.2 parte de uma distinção simples: objetivos descrevem resultados; skills ensinam comportamentos; tools executam operações; o modelo toma decisões semânticas; e o runtime detém a evidência operacional. Para cada objetivo, o runtime recupera skills usando o body completo, seleciona um bundle ordenado de zero a várias instruções e monta um menu composto por  $T_{\text{base}}$  e pelas tools declaradas no bundle. O planner recebe uma única passagem de feedback do catálogo antes da execução e, quando necessário, uma revisão localizada alimentada pelo conjunto completo de observações do nó.

A revisão 0.2 torna explícitos os contratos de catálogo, plano, estado, bundle, contexto, decisão semântica, observação e resultado materializado do nó. Ela retira referências de observação da saída do modelo, mantém `callId` apenas na correlação interna, associa toda revisão às evidências completas do nó e preserva a projeção causal e a composição final única. Essas decisões aumentam a implementabilidade sem acrescentar subsistemas à tese. Não há compilador semântico obrigatório, tool router independente, camada de providers, resolvidor de recursos, solver de bundles, verificador separado ou protocolo operacional completo. A especificação assume execução YOLO e explicita seus limites. Com essa redução, a tese arquitetural pode ser enunciada de forma direta:

*A granularidade do plano deve seguir resultados intermediários; a granularidade das instruções e das operações deve permanecer composicional dentro de cada objetivo.*

## REFERÊNCIAS

AGENT SKILLS. **Specification**. [S. l.], 2026. Disponível em: <https://agentskills.io/specification>. Acesso em: 29 jul. 2026. BABU, Rahul Suresh; IYER, Laxmi-priya Ganesh. ToolMenuBench: benchmarking tool-menu filtering strategies for reliable and efficient LLM agents. **arXiv**, 2026. DOI: 10.48550/ar-Xiv.2606.15508. CHO, Hongcheol; KANG, Ryangkyung; KIM, Youngeun. SkillRet: a large-scale benchmark for skill retrieval in LLM agents. **arXiv**, 2026. DOI: 10.48550/arXiv.2605.05726. GAO, Xueping. Compositional skill routing for LLM agents: decompose, retrieve, and compose. **arXiv**, 2026. DOI: 10.48550/arXiv.2606.18051. GONG, Jingzhi et al. SkillMOO: multi-objective optimization of agent skills for software engineering. **arXiv**, 2026. DOI: 10.48550/arXiv.2604.09297. LI, Xi-

angyi et al. SkillsBench: benchmarking how well agent skills work across diverse tasks. **arXiv**, 2026. DOI: 10.48550/arXiv.2602.12670. SCHICK, Timo et al. Toolformer: language models can teach themselves to use tools. In: ADVANCES IN NEURAL INFORMATION PROCESSING SYSTEMS, 36., 2023. **Proceedings [...]** [S. l.]: Curran Associates, 2023. Disponível em: <https://arxiv.org/abs/2302.04761>. Acesso em: 29 jul. 2026. YAO, Shunyu et al. ReAct: synergizing reasoning and acting in language models. In: INTERNATIONAL CONFERENCE ON LEARNING REPRESENTATIONS, 2023. **Proceedings [...]** [S. l.]: ICLR, 2023. DOI: 10.48550/arXiv.2210.03629. ZHENG, YanZhao et al. SkillRouter: retrieve-and-rerank skill selection for LLM agents at scale. **arXiv**, 2026. DOI: 10.48550/arXiv.2603.22455.

## APÊNDICE A - CONTRATOS NORMATIVOS

Este apêndice define a informação mínima que deve atravessar as fronteiras do runtime. Os Quadros A.1 e A.2 distinguem objetos de catálogo, planejamento, decisão do modelo e evidência criada pelo runtime.

**Quadro A.1 - Contratos de catálogo, plano e routing**

| Objeto         | Campos mínimos                                     | Invariantes                                                                      |
|----------------|----------------------------------------------------|----------------------------------------------------------------------------------|
| SkillRecord    | name, description, body, allowedTools[], indexText | nome único; body não vazio; todas as tools resolvem no registro                  |
| ToolDescriptor | name, inputSchema, outputSchema                    | nome único; schemas disponíveis ao executor                                      |
| GoalNode       | id, goal, doneWhen[], dependsOn[], status, deliver | ID único; dependências válidas; status permitido; objetivo orientado a resultado |
| Plan           | nodes[], revision                                  | DAG acíclico; ao menos um nó entregável; nós concluídos imutáveis em revisões    |
| SkillHint      | goalId, skillName, evidence, effect                | fundamentada no body; effect indica vocabulário, lacuna, divisão ou dependência  |
| SkillCandidate | skillName, score, rank, rationale                  | aponta para skill canônica; rank total e determinístico após desempate           |
| OrderedBundle  | goalId, skills[], selectionRationale               | cardinalidade de zero a $K_{\max}$ ; sem duplicatas; ordem final observável      |

Fonte: elaboração própria (2026).

## Quadro A.2 - Contratos de execução, evidência e revisão

| Objeto          | Autoria e campos mínimos                                                               | Invariantes                                                                                                                           |
|-----------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| NodeContext     | runtime: pedido, objetivo, doneWhen, estado projetado, skills e tools                  | estado causal; bodies delimitados; menu igual a $T(g)$                                                                                |
| NodeDecision    | modelo: status, critérios com evidência em prosa, resultado, pedido de revisão e razão | não contém observações, callId nem referências opacas do provedor                                                                     |
| Observation     | runtime: goalId, toolName, callId, entrada, saída e ordem                              | chamada e retorno correlacionados exatamente uma vez; callId permanece interno; a tool terminal de saída estruturada não é observação |
| NodeOutcome     | runtime: todos os campos de NodeDecision mais observations[]                           | contém cada retorno bem-sucedido de tool executável, na ordem dos resultados                                                          |
| RevisionRequest | modelo: goalId, invalidatedAssumption, requestedEffect                                 | needs_revision exige pelo menos uma observação; o runtime associa automaticamente o conjunto completo produzido pelo nó               |
| FinalDelivery   | runtime: objetivos terminais, partes e referências de artefatos                        | construída sem nova chamada ao modelo; ordem estável e rastreável                                                                     |

Fonte: elaboração própria (2026).

### A.1 Validações normativas

Antes da execução, o runtime deve verificar:

1. unicidade de nomes de skills e tools; 2. resolução de cada item de `allowed-tools`; 3. unicidade de IDs de objetivos; 4. existência de todas as dependências;
5. aciclicidade do plano; 6.  $T_{\text{base}} \subseteq T_{\text{registry}}$ ; 7.  $0 \leq |B(g)| \leq K_{\text{max}}$ ; 8. existência de pelo menos um resultado terminal entregável.

### A.2 Semântica de revisão do nó atual

Quando um nó `running` produz `needs_revision`, o resultado materializado deve conter ao menos uma observação. O runtime associa o pedido semântico a todas as observações do nó e entrega ao planner apenas nome de tool, entrada e saída, preservando a ordem e omitindo `callId`. O planner pode manter o nó, substituí-lo ou inserir predecessores. Se o nó for mantido, seu ID permanece e seu status retorna a `pending`; se for substituído, o ID antigo não pode ser reutilizado. Nenhum resultado parcial é promovido.

## APÊNDICE B - CHECKLIST MOSAIC 0.2

**Quadro B.1 - Requisitos de conformidade da MOSAIC 0.2**

| ID  | Requisito                                                                                                | Obrigatório |
|-----|----------------------------------------------------------------------------------------------------------|-------------|
| C01 | O planner inicial recebe o pedido sem nomes de skills.                                                   | Sim         |
| C02 | Cada nó possui resultado observável, <code>doneWhen</code> e participa de um DAG válido.                 | Sim         |
| C03 | O feedback do catálogo ocorre somente após P0 e no máximo uma vez antes da execução.                     | Sim         |
| C04 | O reranking lê o body canônico das candidatas.                                                           | Sim         |
| C05 | O bundle admite cardinalidade de zero a $K_{\max}$ e ordem determinística.                               | Sim         |
| C06 | O menu é $T_{\text{base}} \cup \bigcup A(s)$ para as skills selecionadas.                                | Sim         |
| C07 | A decisão do modelo contém somente campos semânticos e avaliações ordenadas dos critérios.               | Sim         |
| C08 | O runtime captura automaticamente cada retorno bem-sucedido de tool executável na ordem dos resultados.  | Sim         |
| C09 | Dados de chamada e retorno ausentes, duplicados ou não correlacionados causam falha de runtime.          | Sim         |
| C10 | A tool terminal de saída estruturada nunca é registrada como observação.                                 | Sim         |
| C11 | <code>needs_revision</code> exige ao menos uma observação e associa o pedido ao conjunto completo do nó. | Sim         |
| C12 | O planner localizado recebe nome, entrada e saída de todas as observações, sem <code>callId</code> .     | Sim         |
| C13 | Revisões preservam nós concluídos e seus resultados.                                                     | Sim         |
| C14 | A resposta final é empacotada sem nova chamada cognitiva.                                                | Sim         |

Fonte: elaboração própria (2026).

Extensões podem coexistir com a MOSAIC 0.2 desde que não alterem silenciosamente esses contratos. Quando alterarem, a implementação deve declarar uma versão derivada em vez de usar o rótulo MOSAIC 0.2 sem qualificação.